Точки трансформации системы

Точка трансформации 1. Единый событийный слой заказа и общая история состояний

Бизнес-цель	Дать единую и достоверную картину заказа для поддержки, аналитики и управления. Снизить число ручных разборов и расхождений в данных. Создать основу для поэтапного разделения системы.
Уточнённые требования	Подтверждено: состояния заказа фрагментированы; карточка заказа не является единым источником правды; поддержке нужны ответы «где заказ, что дальше, почему отклонение»; аналитике нужна единая модель заказа и доставки. Уточнено: события должны покрывать заказ, оплату, резерв, доставку, отмены, возвраты и ручные корректировки. Нужны причины отклонений и история изменений.
Ограничения	Нельзя останавливать бизнес. Монолит остаётся главным оркестратором. Oracle содержит часть логики и истории. Ручные состояния и вмешательства существуют. Каналы и внешние площадки имеют разные ветки поведения. Нет утверждённых соглашений об уровне обслуживания и требований к задержке событий.
Препятствия текущего состояния	Часть переходов — в службе заказов, часть — в платёжном модуле, часть — в пакетных процедурах, часть — в административном интерфейсе. Идентификаторы корреляции есть не везде. Аналитика собирает данные из прямых запросов, промежуточных таблиц, программных интерфейсов, пакетной обработки и файлов. Один заказ по-разному выглядит в разных витринах.
Точки трансформации	Управление состояниями, служба обработки заказов, асинхронный и пакетный обмен, аналитические задания, административные и поддержковые инструменты, таблицы истории в Oracle, служба интеграции .NET в части событий. Ввести единые события и коды причин.

Не точки трансформации	Полная замена оформления заказа, вынос платёжной оркестрации, переписывание создания заказа, полная миграция Oracle, отказ от ручных сценариев поддержки на первом этапе.
Приоритизация	Наивысший. Это фундамент для наблюдаемости, поддержки, аналитики и последующего разделения. Без него изменения в других зонах плохо измеримы.
Целевая архитектура первого этапа	Монолит публикует единые доменные события через исходящий буфер или захват изменений. Появляется единая модель состояния заказа и доставки. Формируется модель чтения «история заказа» для поддержки и аналитики. Старые таблицы и пакетная обработка временно сосуществуют.
Разрыв	Процессы: ручные состояния становятся явными исключениями. Данные: появляется единая модель и коды причин. Интеграции: события вместо части прямых чтений. Состояния: состояния нормализуются. Компоненты: управление состояниями, пакетная обработка, аналитика, поддержковые инструменты.
Метрики	Доля заказов с полной историей; задержка обновления состояния; число расхождений между витринами; число эскалаций в поддержку; время ответа поддержки; число ручных корректировок состояния. Базовые и целевые значения не заданы.
Риски и открытые вопросы	Кто владелец единой модели состояний? Как обрабатывать ретроспективные корректировки? Какие требования к согласованности и задержке? Нужны ли отдельные модели для клиента, оператора и аналитики? Кто отвечает за перенос исторических состояний?

Точка трансформации 2. Разгрузка и стабилизация оформления заказа (checkout)

Бизнес-цель	Повысить устойчивость основного потока заказов, предсказуемость на пиках, снизить сбои и потери конверсии. Уменьшить зависимость оформления от тяжёлых синхронных операций.
Уточнённые требования	Подтверждено: служба обработки заказов смешивает расчёт цены, промо, маршрутизацию, состояния, прямые запросы и

	синхронные вызовы. Уточнено: на первом этапе не заменять оформление целиком, а сократить длинную транзакцию, вынести последующую обработку в асинхронный режим, ввести идемпотентность, ограничения по времени, автоматические выключатели и явные политики резервных сценариев.
Ограничения	Платёжную оркестрацию и создание заказа нельзя ломать. Платежи и наличие товара — критичные синхронные зависимости. Производительность БД Oracle близка к пределу. Локальное размещение ограничивает гибкость. Нет значений 95-го и 99-го процентиля, запросов и транзакций в секунду, допустимой задержки оформления.
Препятствия текущего состояния	Длинные транзакции вокруг заказа. Прямые запросы к БД Oracle. Синхронный вызов логистики и платежей при оформлении. Запись в служебные таблицы из кода заказа. Резервный сценарий на унаследованную логику не всегда эквивалентен целевому поведению. Слабая наблюдаемость.
Точки трансформации	Служба обработки заказов, платёжная оркестрация, маршрутизация доставки, асинхронный и пакетный обмен, служебные таблицы выгрузки и повторных попыток, исполнитель унаследованных запросов. Разделить критические и последующие шаги.
Не точки трансформации	Полная замена оформления заказа, вынос платёжной оркестрации, переписывание создания заказа, полная переработка промо-механизма, отказ от Oracle.
Приоритизация	Наивысший. Это напрямую влияет на выручку и стабильность. Но изменения должны быть ограничены разгрузкой и швами, а не перепроектированием ядра.
Целевая архитектура первого этапа	Оформление остаётся в монолите, но транзакция становится короче. Последующие действия — выгрузки, уведомления, часть повторных попыток, аналитические записи — уходят в асинхронный режим. Вводятся ключи идемпотентности, ограничения по времени, автоматические выключатели и явные состояния ожидания. Резервный сценарий описывается как политика с известной деградацией.
Разрыв	Процессы: синхронные шаги становятся асинхронными там, где это безопасно. Данные: появляются состояния ожидания и

	причины. Интеграции: идемпотентность и повторные попытки. Состояния: меньше «зависших» заказов. Компоненты: последующая обработка выделяется из создания заказа.
Метрики	95-й и 99-й процентиль оформления; длительность транзакции; число превышений времени и ошибок оплаты; доля резервных сценариев; число сбоев на пиках; количество заказов в ручном разборе. Базовые и целевые значения не заданы.
Риски и открытые вопросы	Какие зависимости действительно скрыты? Как изменение поведения отличит каналы? Какие целевые уровни обслуживания допустимы? Кто владелец политики резервных сценариев? Как обеспечить откат? Нужны требования платёжной команды и эксплуатации.

Точка трансформации 3. Унификация логистических интеграций и состояний доставки

Бизнес-цель	Сделать обещание доставки надёжнее, снизить число претензий и ручных разборов, уменьшить дубли и зависшие логистические состояния.
Уточнённые требования	Подтверждено: нет единого интеграционного контура; используются веб-службы, очереди, таблицы, пакетная обработка; правила маршрутизации распределены; есть ручные исключения. Уточнено: сначала стабилизировать слой интеграций и состояние доставки, а не выносить маршрутизацию целиком. Нужны идемпотентность, дедупликация, единый договор состояния доставки и события.
Ограничения	Разные партнёры, регионы, каналы, типы доставки. Часть правил интеграции в коде, настройках, Oracle и запросах. Oracle остаётся важным компонентом. Ручные настройки операционного блока используются. Нет актуальной схемы потоков данных.
Препятствия текущего состояния	Смесь синхронного и асинхронного. Партнёр может не подтвердить слот вовремя. Дубли заявок. Разная семантика очередей: гарантированная доставка и опрос таблицы. Обещанный срок доставки меняется после оформления.

Точки трансформации	Шлюз доставки, маршрутизация доставки, асинхронный и пакетный обмен, служба интеграции .NET, промежуточные таблицы Oracle, таблицы приоритетов и коэффициентов, запросы актуализации маршрутов.
Не точки трансформации	Полный вынос маршрутизации, замена всех партнёрских программных интерфейсов, отказ от Oracle, автоматизация всех ручных исключений на первом этапе.
Приоритизация	Наивысший или высокий. Стабилизация интеграций и состояний доставки критична для клиентского опыта и поддержки. Полная маршрутизация — позже.
Целевая архитектура первого этапа	Вводится слой интеграции/изоляции для партнёров. Единый договор состояния доставки. События обновления доставки. Идемпотентные повторные попытки и очередь необработанных сообщений. Реестр исключений и причин ручного разбора. Правила маршрутизации временно остаются в текущих местах, но инвентаризируются.
Разрыв	Интеграции: унификация протоколов и договоров. Данные: единое состояние доставки и причины. Процессы: автоматизация части исключений. Состояния: меньше «ожидает уточнения». Компоненты: слой изоляции, очередь, реестр исключений.
Метрики	Число дублей заявок; число ручных разборов; время подтверждения доставки; доля изменений срока после оформления; ошибки внешних программных интерфейсов; число заказов в состоянии ожидания доставки. Базовые и целевые значения не заданы.
Риски и открытые вопросы	Партнёры могут не менять программные интерфейсы. Скрытые правила маршрутизации в Oracle и запросах. Ручные настройки ночью. Нет соглашений об уровне обслуживания партнёров. Кто владелец единого состояния доставки? Как обрабатывать раздельные и частичные поставки?

Точка трансформации 4. Снижение зависимости от Oracle и обмена через базу данных, отделение аналитической нагрузки

Бизнес-цель	Снизить стоимость развития, подготовить постепенный отказ от Oracle и зарубежных компонентов, уменьшить влияние аналитики на операционный контур, повысить управляемость данных.
Уточнённые требования	Подтверждено: Oracle содержит операционную базу, часть бизнес-логики и интеграционный слой; есть обмен через промежуточные и общие таблицы; аналитика зависит от операционной БД Oracle; архивирование частичное. Уточнено: на первом этапе не мигрировать всё, а остановить новые интеграции через общие таблицы, заменить часть обмена на программные интерфейсы и события, вынести тяжёлые выгрузки, инвентаризировать логику Oracle.
Ограничения	Oracle критична для текущих операций. Служба интеграции .NET и пакетная обработка завязаны на таблицы. Аналитика использует прямые запросы, промежуточные таблицы, программные интерфейсы, пакетную обработку и файлы. Локальное размещение и физические серверы. Нет сроков и бюджета миграции.
Препятствия текущего состояния	Служебные таблицы выгрузки и повторных попыток, промежуточные и общие таблицы. Прямые запросы из монолита. Аналитика на операционной базе. Нет прослеживаемости происхождения данных. Часть логики в процедурах Oracle. Данные десятки терабайт.
Точки трансформации	Операционная база Oracle, исполнитель унаследованных запросов, служба интеграции .NET, аналитические задания, промежуточные и общие таблицы, пакетные процедуры, архивирование.
Не точки трансформации	Полный отказ от Oracle, миграция всех процедур, замена локального размещения, переписывание аналитики целиком.
Приоритизация	Высокий, но для аналитики и производительности может быть наивысшим. Зависит от событийного слоя: без единых событий вынос аналитики будет рискованным.
Целевая архитектура первого этапа	Новые интеграции — только через программные интерфейсы и события. Логика Oracle инвентаризируется. Аналитика читает из реплики или единого хранилища, а не из операционной базы. Тяжёлые выгрузки ограничены и вынесены. Архивирование становится управляемым.

Разрыв	Интеграции: обмен через базу данных заменяется на программные интерфейсы и события. Данные: прослеживаемость происхождения и единый слой. Процессы: архивирование и выгрузки. Компоненты: аналитика отделяется от операционной Oracle.
Метрики	Число интеграций через общие таблицы; нагрузка Oracle от аналитики; объём промежуточных таблиц; время отчётов; сбои из-за аналитики; доля логики, перенесённой из Oracle. Базовые и целевые значения не заданы.
Риски и открытые вопросы	Скрытая логика в Oracle. Аналитика может сломаться. Нет бюджета и сроков. Независимость от зарубежных компонентов может конфликтовать со стабильностью. Кто владелец процедур Oracle? Какие требования к целевому времени и точке восстановления? Какой допустимый период параллельной работы?

Итоговая приоритизация первого этапа

Приоритет	**Точки**	**Обоснование**
Наивысший	Единый событийный слой и история состояний; Разгрузка оформления заказа; Унификация логистических интеграций и состояний доставки	Это зоны, которые напрямую влияют на выручку, клиентский опыт, поддержку и возможность дальнейших изменений. Без них остальные улучшения будут точечными и плохо измеримыми.
Высокий	. Снижение зависимости от Oracle и отделение аналитики	Важны для скорости бизнеса, независимости от зарубежных компонентов и стоимости развития. Но требуют осторожности и опираются на результаты первого приоритета: события, состояния, разгрузку транзакций.

Целевая архитектура первого этапа

Целевая архитектура — это **не переход к набору микросервисов и не полная переработка ядра**. Это монолит, который остаётся главным управляющим звеном, но постепенно получает: единый слой событий, асинхронную последующую обработку, единые адаптеры к внешним системам и отделённый аналитический контур (т.н. границы/швы/seams — места сочленения, стыковки или границы между различными модулями, компонентами или сервисами, в которых можно изменить поведение системы без редактирования самого кода в этом месте) . Всё, что влияет на выручку и обещания клиенту, не переписывается, а разгружается и оборачивается в управляемые правила. Такой подход допустим по риску, потому что не требует одновременной замены ядра, сохраняет текущее поведение и даёт промежуточную пользу на каждом шаге.

Что остаётся в монолите

Часть	Почему остаётся
Оформление заказа и входной программный интерфейс	Слишком высокая связность с правилами, каналами и сессиями. Замена на первом этапе — критический риск для выручки.
Создание заказа	Ядро жизненного цикла. Содержит смесь цены, акций, маршрутизации и состояний. Скрытые зависимости не описаны.
Управление платежами	Прямо влияет на оплату и выручку. Синхронные вызовы платёжных поставщиков.
Выбор маршрута доставки	Правила распределены между кодом, базой, настройками и запросами. Нет актуальной схемы.
Критические переходы состояний	Часть переходов в пакетной обработке, платёжном модуле и административном интерфейсе.
Логика в Oracle	Содержит скрытые правила, особенно в акциях, маршрутизации и интеграциях.

Что изолируется

Часть	Как изолируется	Зачем
Единый слой событий	Монолит публикует доменные события через исходящий буфер или захват изменений	Отделяет потребителей от внутренней модели монолита. Даёт единую историю и наблюдаемость.

Последующая обработка заказа	Выносится из синхронной транзакции в асинхронные обработчики	Сокращает длинную транзакцию, снижает нагрузку на базу в момент оформления.
Логистические интеграции	Создаётся слой изоляции для партнёров. Единый договор состояния доставки, повторные попытки, очередь необработанных сообщений	Убирает смесь способов обмена, снижает дубли и зависшие состояния.
Аналитический контур	Отделяется от операционной базы: реплика или единое хранилище	Снижает влияние аналитики на оформление. Даёт единую картину данных.
Правила акций	Вводится реестр правил и коды причин. Служба акций используется шире, унаследованная логика остаётся как резерв	Делает расчёт скидок объяснимым и управляемым.

Что выносится первыми

- Единые события и история состояний — фундамент для всего остального. Без них нельзя измерить эффект и дать поддержке единую картину.
- Асинхронная последующая обработка заказа — разгружает оформление, снижает нагрузку на базу на пиках.
- Унификация логистических интеграций и состояний доставки — напрямую влияет на клиентский опыт и число ручных разборов.

  Эти направления не требуют немедленного выноса критичных частей. Они создают швы, через которые позже можно безопасно выделять отдельные службы.

Где проходят границы

- **Граница монолита:** оформление, создание заказа, управление платежами, маршрутизация, критические переходы состояний.

- **Граница изоляции:** слой событий, последующая обработка, логистические адаптеры, аналитический контур, реестр правил акций.
- **Граница данных:** операционная база остаётся для транзакций, но новые интеграции и аналитика идут через программные интерфейсы и события. Обмен через общие таблицы для новых сценариев запрещается.
- **Граница ответственности:** монолит по-прежнему принимает решения по заказу, но делегирует побочные действия и интеграции изолированным частям.

Почему изменения допустимы по риску

- **Критичный контур не трогаем:** оформление, управление платежами и создание заказа остаются в монолите без переписывания.
- **Поэтапность:** каждый шаг даёт промежуточную пользу и может быть откачен.
- **Текущее поведение сохраняется:** резервные сценарии, ручные операции и логика Oracle продолжают работать, но становятся явными и логируемыми.
- **Управляемая деградация:** ограничения по времени, автоматические выключатели, повторные попытки и очередь необработанных сообщений — стандартные приёмы, не требующие изменения бизнес-логики.
- **Опора на текущие данные:** единые события строятся на основе существующих таблиц и истории без потери данных.

Итог

Целевая архитектура первого этапа — это **монолит с выделенными швами и единым слоем событий**. Монолит остаётся ядром, но перестаёт быть единственным источником правды для аналитики, поддержки и интеграций. Первыми выносятся наименее рискованные и наиболее ценные для наблюдаемости и стабильности участки. Критичные для выручки и обещаний клиенту части не переписываются, а разгружаются и оборачиваются в управляемые правила. Такой подход даёт промежуточную пользу, снижает риски и создаёт основу для дальнейшего поэтапного разделения.